



Pets Expression Classification

Computer Vision

Assignment 4

Report

Faculty of Engineering

Alexandria University

Prepared By

Ahmad Yasser Hamad

9074

Moamen Talaat

9103

Contents

1	Introduction	2
2	Dataset Description and Preprocessing	2
2.1	Dataset Overview	2
2.2	Data Splits	3
2.3	Data Augmentation	3
3	Building Networks from Scratch	3
3.1	Common Training Setup	4
3.2	Optimizer and Batch Size	4
3.3	VGG16	5
3.4	ResNet18	7
3.5	MobileNet	8
3.6	Inception V3	10
3.7	DenseNet121	12
3.8	Comparison of From-Scratch Models	14
4	Transfer Learning	15
4.1	Motivation	15
4.2	Methodology	15
4.3	Results	17
5	Conclusion	19

1 Introduction

The goal of this assignment was to build and evaluate deep convolutional neural networks (CNNs) capable of recognizing the facial expressions of pets. The provided dataset contains face images of various pets, each displaying one of four emotional expressions. The pets included in the dataset include cats, dogs, hamsters, rabbits, sheep, horses, and birds. The emotional expressions considered are either happy, sad, angry, or another out-of-scope expression. Automatically classifying these expressions can help owners better understand the emotional state and well-being of their pets.

The assignment is divided into two parts. In the first part, five well-known CNN architectures — VGG16, ResNet18, MobileNet, Inception V3, and DenseNet121 — are implemented from scratch and trained on the dataset with data augmentation. In the second part, transfer learning is applied: the best-performing architecture from the first part is re-built from its standard library implementation and initialized with ImageNet pre-trained weights instead of a random initialization, and the resulting improvement is reported.

For every model we report the test accuracy, the mean (macro-averaged) precision, recall, and F1-score, together with its confusion matrix.

2 Dataset Description and Preprocessing

2.1 Dataset Overview

The dataset consists of RGB face images of pets divided into four expression classes. The training set is balanced, containing 250 images per class.

Table 1: Expression classes and per-subset image counts.

Subset	Number of Images
Training set	1000 (250 per class)
Validation set	36
Test set	38

The data-loading utilities infer the label of each image from its parent directory (the name of the folder in which the image is included) and order the classes alphabetically, giving the index mapping 0 → Angry, 1 → Other, 2 → Sad, 3 → Happy. This pattern is found in all confusion matrices.

2.2 Data Splits

The dataset was divided into three disjoint subsets: a training set used to fit the model parameters, a validation set used to monitor generalization during training, and a test set used for the final unbiased evaluation. It is worth noting that the validation and test sets are very small (36 and 38 images respectively); consequently their metrics are noisy, and a single misclassified image largely disturbs the measured accuracy.

2.3 Data Augmentation

As required, data augmentation was applied during training. Augmentation artificially enlarges the effective size of the small training set by generating randomly transformed copies of the input images, which reduces overfitting and improves generalization. Augmentation is applied *only* to the training set; the validation and test sets are merely resized and normalized so that evaluation reflects the true, untransformed data distribution.

Two deep-learning frameworks were used across the assignment, as permitted by the requirements: the from-scratch VGG16, ResNet18, and MobileNet models, together with the transfer-learning model, were implemented in **PyTorch**, while DenseNet121 and Inception V3 were implemented in **Keras**. The difference stemmed from our willingness to explore multiple methods through which such architectures can be tackled. The augmentation pipelines of the two frameworks are summarized below.

Table 2: Training-time data augmentation per framework.

PyTorch (VGG16, ResNet18, MobileNet, Transfer Learning)	Keras (DenseNet121, Inception V3)
Random resized crop	Rotation range 20°
Random horizontal flip	Width / height shift 0.2
Random rotation 15°	Shear range 0.2
Color jitter 0.2	Zoom range 0.2
ImageNet mean/std normalization	Horizontal flip
	Rescaling 1/255

3 Building Networks from Scratch

In this part, each architecture is implemented layer-by-layer and trained from a random weight initialization. No pre-trained weights are used.

3.1 Common Training Setup

All models were trained for 20 epochs with a batch size of 32, using the Adam optimizer and a cross-entropy loss. The only structural difference between experiments is the architecture itself and the input resolution (Inception V3 requires 299×299 inputs, while the others use 224×224).

3.2 Optimizer and Batch Size

Adam optimizer. All models were trained with the *Adam* (Adaptive Moment Estimation) optimizer. Adam combines two ideas: *momentum*, which accumulates an exponentially decaying average of past gradients, and *RMSProp*, which scales the step of each parameter by an exponentially decaying average of past squared gradients. It therefore maintains a separate, adaptive learning rate for every parameter. At each step it updates a first moment estimate m_t and a second moment estimate v_t :

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2,$$

where g_t is the current gradient. After a bias-correction step that compensates for the zero-initialization of m_t and v_t , the parameters θ are updated as

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}},$$

with η the learning rate and ϵ a small constant for numerical stability. Adam was chosen because it converges quickly, adapts the learning rate per parameter automatically, and performs well with little manual tuning — a desirable property given the small dataset.

Batch size. A batch size of 32 was used for all experiments. The batch size controls how many images contribute to each gradient estimate, and its choice is a trade-off: very large batches give smooth but memory-expensive gradient estimates, while very small batches give noisy estimates and unstable batch-normalization statistics. A value of 32 is a widely used compromise — it is small enough to introduce a mild gradient noise that aids generalization, yet large enough to produce stable batch-normalization statistics, and it fits comfortably in GPU memory even for the deepest model (Inception V3 at 299×299 resolution).

Table 3: Per-model training configuration for Part 1.

Model	Framework	Input Size	Optimizer	Learning Rate
VGG16	PyTorch	224×224	Adam	10^{-4}
ResNet18	PyTorch	224×224	Adam	10^{-4}
MobileNet	PyTorch	224×224	Adam	10^{-4}
Inception V3	Keras	299×299	Adam	default
DenseNet121	Keras	224×224	Adam	default

3.3 VGG16

VGG16 is a deep but architecturally simple network: every convolution is a 3×3 “same” convolution followed by a ReLU activation, and each stage is closed by a 2×2 max-pooling layer. The feature extractor follows the configuration $[64, 64, M, 128, 128, M, 256, 256, 256, M, 512, 512, 512, M, 512, 512, 512, M]$, where “M” denotes a max-pooling layer. The classifier head flattens the feature maps and passes them through two fully connected layers of 4096 units (each followed by ReLU and dropout) before the final output layer.

Training behaviour. VGG16 failed to learn from scratch. As shown in Table 4, both the training and validation loss remained essentially frozen at $\ln(4) \approx 1.386$ — the loss of random guessing — for the entire 20 epochs. VGG16 has no batch normalization, which makes such a deep network extremely difficult to optimize from a random initialization on a small dataset; the gradients effectively vanished and no useful learning occurred.

Table 4: VGG16 — training progression (selected epochs).

Epoch	Train acc.	Train loss	Val. acc.	Val. loss
1	0.233	1.388	0.167	1.389
5	0.254	1.386	0.306	1.386
10	0.232	1.387	0.167	1.387
15	0.238	1.387	0.250	1.386
20	0.238	1.387	0.306	1.386

Table 5: VGG16 — test-set classification report.

Class	Precision	Recall	F1-score	Support
Angry	0.00	0.00	0.00	10
Other	0.00	0.00	0.00	6
Sad	0.00	0.00	0.00	11
Happy	0.29	1.00	0.45	11
Accuracy		0.29		
Macro average	0.07	0.25	0.11	38
Weighted average	0.08	0.29	0.13	38

The confusion matrix in Figure 1 confirms the failure: the model assigns every test image to the “Happy” class.

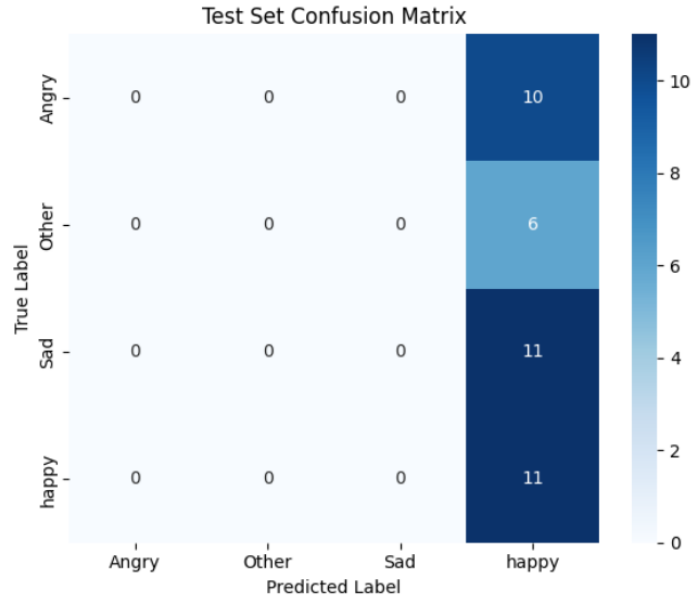


Figure 1: Confusion matrix — VGG16 (from scratch).

Why every prediction collapses to “Happy”. This behavior is the direct symptom of a feature extractor that has never been trained. Because VGG16 has no batch normalization, the gradients vanished while propagating back through its very deep stack of convolutions, so the convolutional layers stayed close to their random initialization and produced an essentially constant, input-independent feature vector for every image. The classifier head therefore receives almost the same input regardless of which pet is shown, and its softmax

output is nearly identical for all images — making the predicted class (the $\arg \max$ of that output) the same for the entire test set.

Which class “wins” is not meaningful. The training set is perfectly balanced (250 images per class), so no class-frequency bias pushes the network toward any particular label; the constant output vector is shaped only by the random weight initialization and the tiny, noisy drift of the bias terms during the failed optimization. “Happy” simply happened to receive the largest logit in this run, and a different random seed would collapse the model onto a different class. The resulting test accuracy of 0.29 is exactly the proportion of “Happy” images in the test set (11/38), confirming that the model is doing nothing more than guessing a single fixed label.

3.4 ResNet18

ResNet18 introduces residual (skip) connections that add the input of a block to its output, allowing gradients to flow more easily through deep networks. Each basic residual block contains two 3×3 convolutions with batch normalization; when the spatial size or channel count changes, the shortcut is projected with a 1×1 convolution so that the addition remains dimensionally valid. The network begins with a 7×7 stride-2 convolution and max-pooling, followed by four residual stages of two blocks each (with 64, 128, 256, 512 channels), and ends with global average pooling and a fully connected classifier.

Training behaviour. Unlike VGG16, ResNet18 trained successfully: thanks to its residual connections and batch normalization, the training accuracy rose steadily from 0.25 to 0.52. The validation accuracy, however, fluctuated heavily — a direct consequence of the very small (36-image) validation set and the limited training data.

Table 6: ResNet18 — training progression (selected epochs).

Epoch	Train acc.	Train loss	Val. acc.	Val. loss
1	0.253	1.464	0.306	1.388
5	0.339	1.351	0.278	1.460
10	0.428	1.261	0.222	1.640
15	0.481	1.185	0.417	1.439
20	0.515	1.099	0.333	1.448

Table 7: ResNet18 — test-set classification report.

Class	Precision	Recall	F1-score	Support
Angry	0.30	0.30	0.30	10
Other	0.12	0.17	0.14	6
Sad	0.75	0.27	0.40	11
Happy	0.44	0.64	0.52	11
Accuracy				0.37
Macro average	0.40	0.34	0.34	38
Weighted average	0.44	0.37	0.37	38

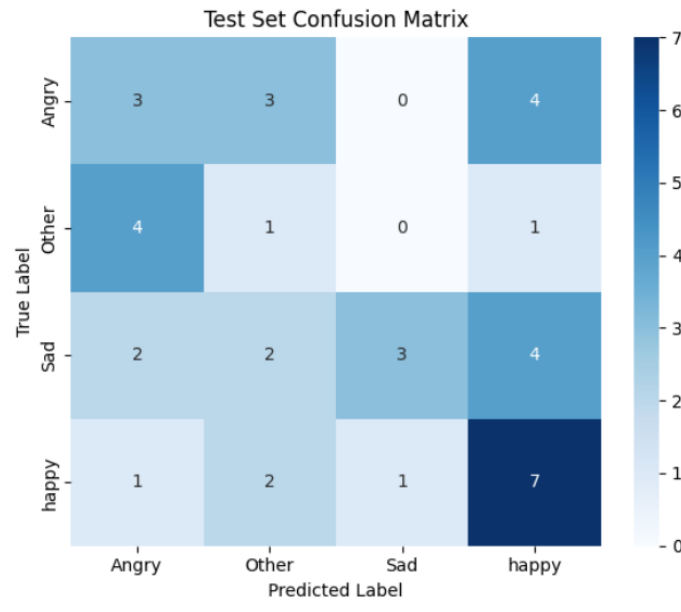


Figure 2: Confusion matrix — ResNet18 (from scratch).

With a test accuracy of 0.37, ResNet18 was the **best-performing from-scratch architecture** and was therefore selected for the transfer-learning part.

3.5 MobileNet

MobileNet is designed for efficiency. It replaces standard convolutions with depthwise separable convolutions: a depthwise 3×3 convolution that filters each channel independently, followed by a pointwise 1×1 convolution that mixes the channels. Both steps are followed by batch normalization and ReLU. This factorization drastically reduces the

number of parameters and computations while retaining most of the representational power of a standard convolution.

Training behaviour. MobileNet learned only weakly: the training accuracy crept up to roughly 0.34 and the validation accuracy never stabilized, reflecting the difficulty of fitting the architecture on only 1000 images.

Table 8: MobileNet — training progression (selected epochs).

Epoch	Train acc.	Train loss	Val. acc.	Val. loss
1	0.264	1.391	0.167	1.440
5	0.293	1.376	0.250	1.365
10	0.355	1.345	0.278	1.374
15	0.325	1.366	0.278	1.465
20	0.339	1.352	0.250	1.477

Table 9: MobileNet — test-set classification report.

Class	Precision	Recall	F1-score	Support
Angry	0.28	0.80	0.41	10
Other	0.20	0.17	0.18	6
Sad	1.00	0.09	0.17	11
Happy	0.33	0.09	0.14	11
Accuracy	0.29			
Macro average	0.45	0.29	0.23	38
Weighted average	0.49	0.29	0.23	38

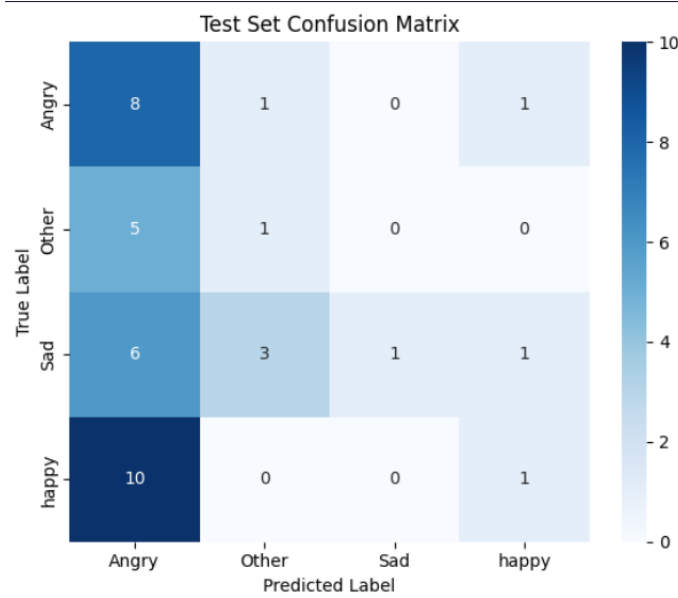


Figure 3: Confusion matrix — MobileNet (from scratch).

3.6 Inception V3

Inception V3 builds the network from Inception modules, which process the same input in parallel through several branches of differing receptive-field sizes (1×1 , factorized 3×3 , factorized 7×7 , and pooling) and concatenate the results. Factorizing large convolutions (for example, decomposing a 7×7 convolution into a 1×7 followed by a 7×1) reduces the parameter count while preserving the receptive field. Our implementation uses a convolutional stem, three Inception-A modules, a Reduction-A module, four Inception-B modules, a Reduction-B module, two Inception-C modules, global average pooling and the final softmax classifier. Inception V3 expects an input resolution of 299×299 .

Training behaviour. Trained from scratch for 20 epochs, the model showed a slowly rising training accuracy and a highly unstable validation loss, including an extreme transient spike at epoch 10. Despite the instability, the resulting predictions on the test set were noticeably better balanced across the four classes than for the other Keras-based from-scratch model (DenseNet121), which is reflected in a higher macro F1-score.

Table 10: Inception V3 — training progression (selected epochs).

Epoch	Train acc.	Train loss	Val. acc.	Val. loss
1	0.269	1.783	0.194	1.398
5	0.343	1.350	0.194	1.553
10	0.367	1.317	0.167	1519.67
15	0.381	1.307	0.389	2.603
20	0.446	1.231	0.361	2.832

Table 11: Inception V3 — test-set classification report.

Class	Precision	Recall	F1-score	Support
Angry	0.31	0.40	0.35	10
Other	0.67	0.33	0.44	6
Sad	0.22	0.18	0.20	11
Happy	0.23	0.27	0.25	11
Accuracy	0.29			
Macro average	0.36	0.30	0.31	38
Weighted average	0.32	0.29	0.29	38

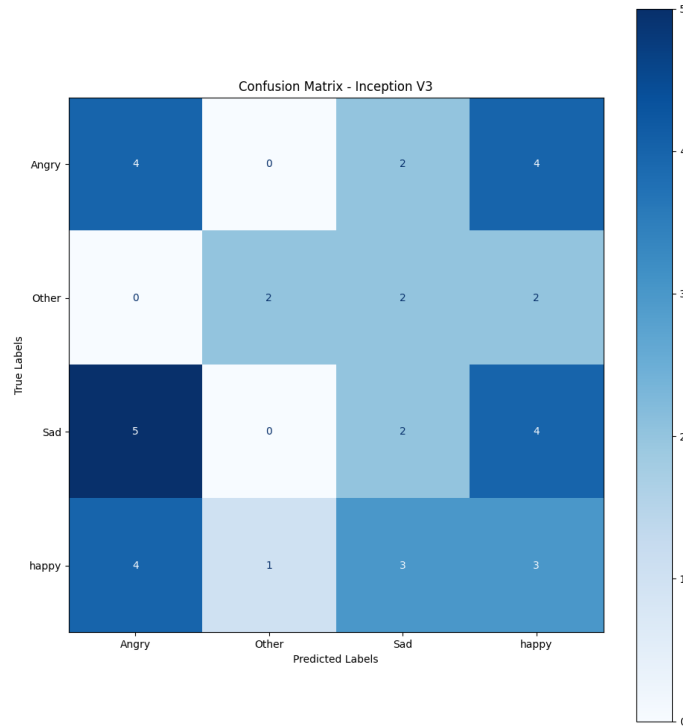


Figure 4: Confusion matrix — Inception V3 (from scratch).

3.7 DenseNet121

DenseNet121 uses dense connectivity: within a dense block, every layer receives, as input, the concatenated feature maps of all preceding layers. This encourages feature reuse, strengthens gradient flow, and keeps the parameter count low. Each composite layer applies batch normalization, ReLU, and convolution (a 1×1 bottleneck followed by a 3×3 convolution). Between dense blocks, transition layers halve the number of feature maps and the spatial resolution. The network uses four dense blocks with $[6, 12, 24, 16]$ layers and a growth rate of 32, followed by global average pooling and a softmax classifier.

Training behaviour. Trained from scratch for 20 epochs, DenseNet121 displayed the same underfitting pattern: the training accuracy climbed gradually while the validation accuracy fluctuated, never stabilizing.

Table 12: DenseNet121 — training progression (selected epochs).

Epoch	Train acc.	Train loss	Val. acc.	Val. loss
1	0.266	1.507	0.306	1.386
5	0.341	1.345	0.278	1.427
10	0.384	1.309	0.278	1.438
15	0.424	1.269	0.417	1.583
20	0.437	1.244	0.222	2.222

Table 13: DenseNet121 — test-set classification report.

Class	Precision	Recall	F1-score	Support
Angry	0.29	1.00	0.45	10
Other	0.00	0.00	0.00	6
Sad	1.00	0.09	0.17	11
Happy	0.00	0.00	0.00	11
Accuracy	0.29			
Macro average	0.32	0.27	0.16	38
Weighted average	0.37	0.29	0.17	38

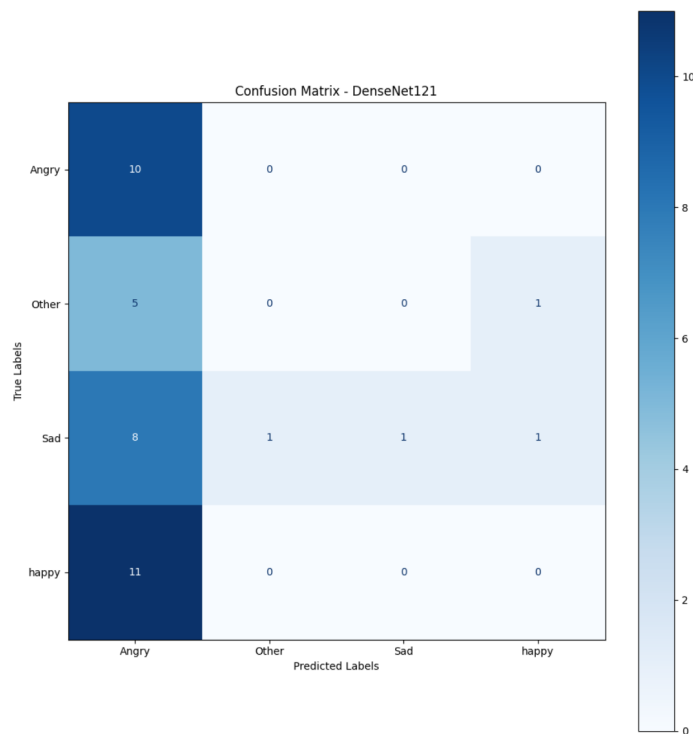


Figure 5: Confusion matrix — DenseNet121 (from scratch).

3.8 Comparison of From-Scratch Models

Table 14 collects the test-set results of all five architectures for direct comparison.

Table 14: Test-set comparison of the five from-scratch architectures (macro-averaged metrics).

Architecture	Accuracy	Precision	Recall	F1-score
VGG16	0.29	0.07	0.25	0.11
ResNet18	0.37	0.40	0.34	0.34
MobileNet	0.29	0.45	0.29	0.23
Inception V3	0.29	0.36	0.30	0.31
DenseNet121	0.29	0.32	0.27	0.16

Discussion. All five from-scratch models reached only modest accuracy, between 0.29 and 0.37 — barely above the 0.25 random-guess baseline for a four-class problem. This is the expected outcome: deep CNNs contain millions of parameters, and a training set of only 1000 images is far too small to fit them from a random initialization. Several specific observations stand out:

- **VGG16** failed completely. Without batch normalization, the very deep stack of convolutions could not be optimized from scratch, and the loss never moved from its random-guess value. The model collapsed to predicting a single class for all inputs.
- **ResNet18** performed best. Its residual connections and batch normalization allowed gradients to flow, so it was the only model whose training accuracy climbed convincingly (to 0.52), and it achieved the best test accuracy (0.37).
- **MobileNet, Inception V3, and DenseNet121** all underfit, hovering around chance level on the test set. MobileNet and DenseNet121 collapsed onto one or two classes, while Inception V3 produced more balanced (though still weak) predictions across all four classes — giving it the second-best macro F1-score in this part despite tying the lowest accuracy.

This limitation is precisely what motivates the transfer-learning approach in the next section. Since **ResNet18** was the best-performing architecture, the ResNet family was carried forward to Part 2.

4 Transfer Learning

4.1 Motivation

Training a deep network from scratch requires a large amount of labeled data. When only a small dataset is available — as is the case here — a far more effective strategy is transfer learning: starting from weights that were pre-trained on a large dataset (ImageNet, with 1.2 million images across 1000 classes) and adapting them to the target task. The pre-trained convolutional layers already encode general-purpose visual features (edges, textures, shapes, parts), so the network does not have to learn them from the 1000 pet images. Because ResNet18 was the best-performing architecture in Part 1, the same architecture is reused here, this time initialized from ImageNet weights using the standard `torchvision` implementation. Using the identical architecture isolates the effect of the pre-trained weights themselves — any improvement over Part 1 can be attributed directly to transfer learning rather than to a change of model.

4.2 Methodology

Model construction. The pre-trained ResNet18 backbone is loaded from `torchvision.models` with the `IMAGENET1K_V1` weights. The entire backbone is then *frozen* (every parameter set to `requires_grad=False`), and the original 1000-class `fc` layer is replaced by a small new head sized for the four expression classes. A

Dropout(0.3) layer in front of the new Linear discourages the head from memorizing the small training set.

```
from torchvision.models import resnet18, ResNet18_Weights

model = resnet18(weights=ResNet18_Weights.IMAGENET1K_V1)
for param in model.parameters():
    param.requires_grad = False                # freeze the backbone
model.fc = nn.Sequential(
    nn.Dropout(0.3),
    nn.Linear(model.fc.in_features, 4)          # 4 expression classes
)
```

Preprocessing. The training images go through the same PyTorch augmentation pipeline as the from-scratch models (random resized crop, horizontal flip, rotation, color jitter, and ImageNet mean/std normalization); the validation and test sets are resized, center-cropped, and normalized in exactly the same way the ImageNet ResNet18 was trained, so the frozen backbone always sees the input distribution it expects.

Two-phase training. Training proceeds in two phases so that the pre-trained features are first leveraged and then gently refined:

1. **Phase 1 — head only.** The backbone stays frozen and only the new classifier head is trained, using Adam at a reduced learning rate of 10^{-4} . This lets the head quickly fit a good linear classifier on top of the ImageNet features without disturbing them.
2. **Phase 2 — fine-tuning.** The deepest residual stage (layer4) and the new fc head are unfrozen, while every earlier layer remains frozen so that the generic low-level features (edges, textures) are preserved. The learning rate is dropped tenfold to 10^{-5} so that the pre-trained features are gently adapted rather than overwritten. Crucially, every BatchNorm layer is forced to remain in `eval()` mode throughout this phase — letting it switch back to training mode would update its moving statistics with the small dataset’s batch statistics and destabilize the entire backbone.

Regularization. Several regularization techniques were combined to counter overfitting on the small dataset: dropout in the classifier head, label smoothing ($\epsilon = 0.1$) in the cross-entropy loss to discourage over-confident predictions, a ReduceLROnPlateau learning-rate scheduler that halves the learning rate when the validation loss plateaus, and an early-stopping mechanism that monitors the validation loss and restores the best weights once it stops improving.

Table 15: Transfer-learning configuration.

Setting	Value
Framework / Backbone	PyTorch / torchvision.models.resnet18 (ImageNet weights)
Classifier head	Dropout(0.3) → Linear(512, 4)
Phase 1 (head only)	Adam, lr = 10^{-4} , backbone frozen
Phase 2 (fine-tuning)	Adam, lr = 10^{-5} , layer4 + fc unfrozen, BN kept in eval()
Loss function	Cross-entropy with label smoothing 0.1
Callbacks	ReduceLROnPlateau (factor 0.5, patience 5), early stopping
Epochs / Batch size	up to 25 + 15 / 32

4.3 Results

Table 16: Transfer learning (ResNet18, ImageNet weights) — test-set classification report.

Class	Precision	Recall	F1-score	Support
Angry	0.64	0.70	0.67	10
Other	0.50	0.17	0.25	6
Sad	0.47	0.82	0.60	11
Happy	0.67	0.36	0.47	11
Accuracy	0.55			
Macro average	0.57	0.51	0.50	38
Weighted average	0.58	0.55	0.52	38

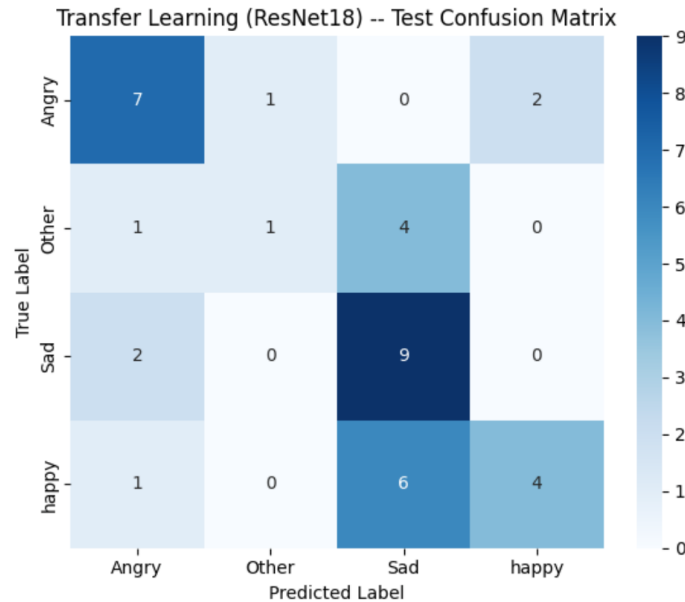


Figure 6: Confusion matrix — transfer learning with ResNet18 (ImageNet weights).

Table 17: Transfer learning vs. training the same architecture from scratch.

Metric	ResNet18 (from scratch)	ResNet18 (transfer learning)
Test accuracy	0.37	0.55
Mean precision (macro)	0.40	0.57
Mean recall (macro)	0.34	0.51
Mean F1-score (macro)	0.34	0.50

Discussion. Transfer learning produced a clear and substantial improvement. The test accuracy rose from 0.37 (the same ResNet18 architecture trained from scratch) to 0.55, and every macro-averaged metric improved by a comparable margin — the F1-score climbed from 0.34 to 0.50 and the macro precision from 0.40 to 0.57. Because the ImageNet-pre-trained ResNet18 backbone already provides rich, general-purpose visual features, the network only had to learn a small classifier head and then gently refine its deepest stage, both of which are feasible even with 1000 images. The confusion matrix in Figure 6 shows that, unlike the from-scratch models, the transfer-learning model produces meaningful predictions across all four classes rather than collapsing onto one. The weakest class remains “Other”, which is expected: it is the least visually consistent category, and with only 6 test images its metrics are also the noisiest. Overall, the experiment confirms that leveraging pre-trained features is the decisive factor when working with a small dataset —

swapping a random initialization for ImageNet weights, with no change to the underlying architecture, was enough to almost double the macro F1-score.

5 Conclusion

In this assignment, five well-known CNN architectures — VGG16, ResNet18, MobileNet, Inception V3, and DenseNet121 — were implemented from scratch and trained to classify pet facial expressions into four classes. As expected, all five struggled to generalize from the small 1000-image training set, achieving test accuracies between 0.29 and 0.37. VGG16 failed to optimize at all without batch normalization, while ResNet18 — aided by its residual connections — was the best performer at 0.37 accuracy.

Transfer learning was then applied using the *same* ResNet18 architecture, this time initialized from ImageNet pre-trained weights. A two-phase schedule was used — first training only a new dropout-regularized classifier head with the backbone frozen, then fine-tuning the deepest residual stage at a much smaller learning rate while keeping every BatchNorm layer frozen — together with label smoothing, a ReduceLROnPlateau scheduler, and early stopping. This raised the test accuracy from 0.37 (random initialization) to 0.55 and the macro F1-score from 0.34 to 0.50. The results clearly demonstrate that, for small datasets, leveraging pre-trained features through transfer learning is far more effective than training a deep network from a random initialization: keeping the architecture identical and swapping only the initial weights was enough to almost double the macro F1-score.